

Illegal Parking Detection with YOLO

*Single-class YOLO detector trained on illegal parking
(positives) + car-centered background negatives.*

<https://github.com/Willem500i/introml/tree/master/project>

Willem Neufeind Lessig

Problem & Goal

Illegally or idling parked cars block lanes, reduce visibility, and increase crash risk. I personally had a crash in Boston due to cars idling in active lanes—this is a real safety issue.

Challenge: public datasets rarely label specific parking violations; we lack per-violation ground truth.

Scoped goal: single-class “illegal parking” vs background, with realistic negatives to avoid flagging everything.

He pulled out behind me while I was driving by an idling car :/

He had just turned 18 and was visiting from new hampshire, gonna have some pretty high insurance rates now sadly.



Data Journey & Sources

Sources:

- Roboflow “Illegal Parking” (positives, car-centered): <https://app.roboflow.com>
- BDD100K (background car crops for negatives): <https://www.bdd100k.com>

Tried: BDD100K alone (no parking labels), other public sets (no violation labels), Roboflow positives only.

Final composition: Roboflow positives + BDD100K car-centered negatives.

Splits: train 1226 (576 pos / 650 neg), val 150 (75/75), test 80 (40/40).



illegal parking (39).png



illegal parking (197).png



illegal parking (179).png



illegal parking (95).png



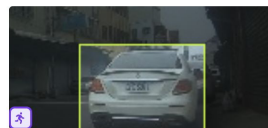
illegal parking (231).png



illegal parking (67).png



illegal parking (330).png



illegal parking (739).png



illegal parking (613).png

Preprocessing & Balancing

Positives: Roboflow illegal-parking images, car-centered $\sim 640 \times 640$.

Negatives: crops from BDD100K car/truck/bus boxes with padding, resized to 640×640 , empty labels $\rightarrow$ background.

Balanced merges; moved some positives into val to ensure labeled validation; single class ["illegal parking"].

Augmentations (YOLOv8 defaults): horizontal flip ($p=0.5$), color jitter, mosaic; consistent framing for both pos/neg to reduce false positives.

Positive $\rightarrow$

(illegal parking set)



$\leftarrow$ Negative

(Cropped from 100k driving images)

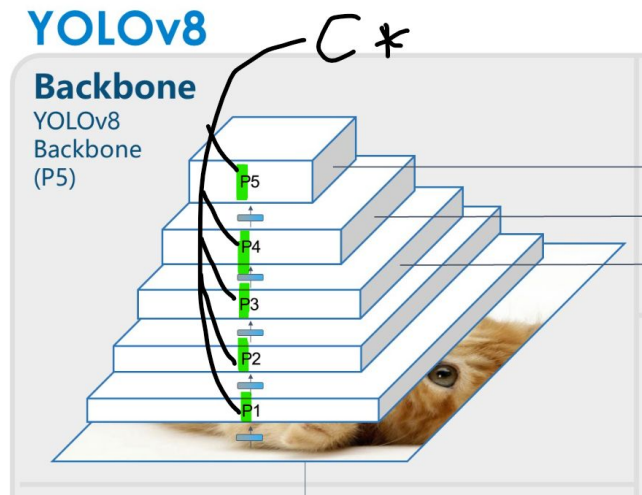


Model Choice & Architecture

YOLOv8n: ~3.0M params, ~8.2 GFLOPs, 72 layers;
pretrained backbone; head overridden to nc=1.

Why: fast iteration on CPU, single class for clear
signal, consistent 640×640 framing; 3 detection
scales handle object size variation.

Could step up to yolov8s if we need more recall and
can afford latency.



YOLOv8 github:

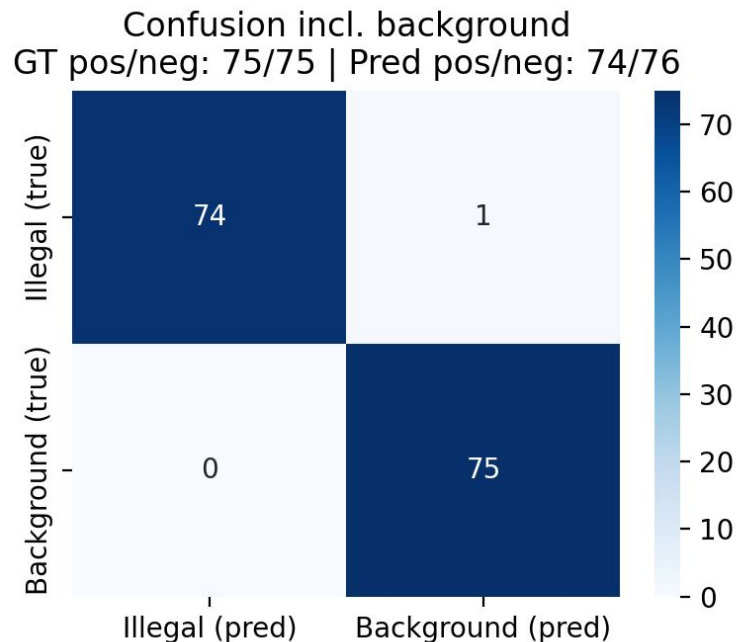
<https://github.com/ultralytics/ultralytics/issues/189>

Results - pretty good!

Val (150 imgs: 75 pos / 75 neg): Precision 0.996,
Recall 0.987, mAP50 0.994, mAP50-95 0.782.

Background-inclusive confusion (conf=0.25): TP 74,
FP 0, FN 1, TN 75. Interpretation: extremely few
misses, no false positives at this threshold.

Built-in confusion (foreground-only) available but
less informative about background.

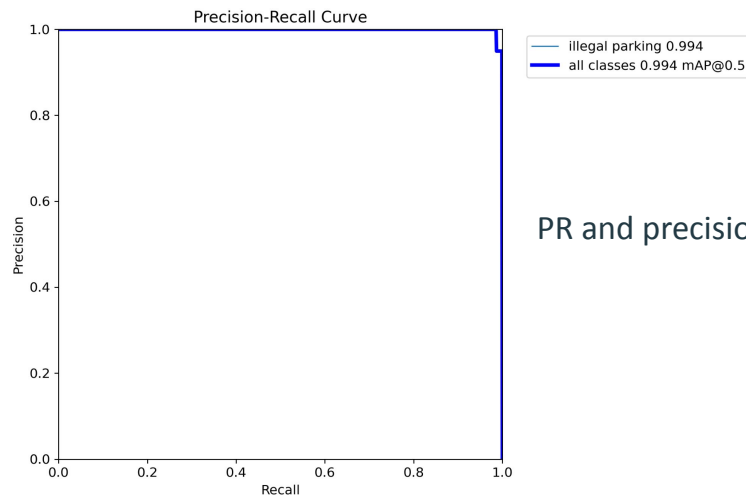
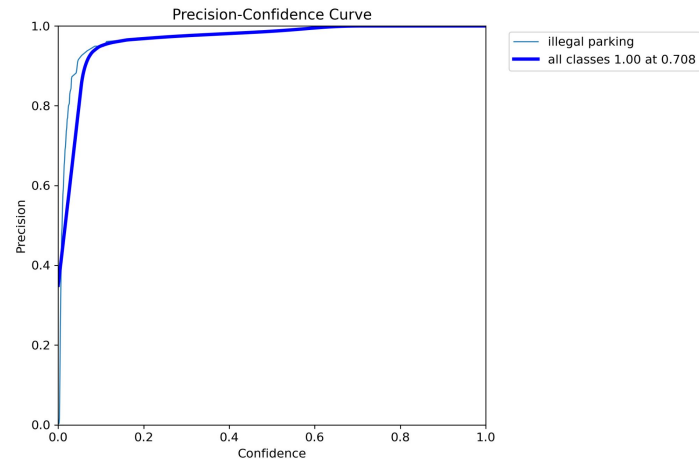


Curves & Trends

PR and precision curves (boxpr_ep5.png, boxp_ep5.png) show very high precision across recall levels.

Training/val curves (results_ep5.png) show stable convergence by 5 epochs, minimal overfit.

Takeaway: current model is already strong; further training offers diminishing returns without better data/labels.



PR and precision curves

Qualitative Examples

The model cleanly flags centered illegally parked cars while leaving visually similar background car crops untouched, showing it learned the pos/neg framing we engineered.

On `val_batch0_pred_ep5.jpg`, detected boxes align well with the ground truth in `val_batch0_labels_ep5.jpg`, illustrating tight localization and high confidence.

Background-only samples (legal parked or lane cars) are ignored at the chosen threshold, reducing false alarms and supporting reliable ticketing evidence.



Ticketing / Evidence Workflow

Inference: yolo detect predict with save_txt, save_conf, save_crop.

Export: export_detections.py → JSONL (image, bbox, confidence, crop path).

Evidence page: ticket_example_ep5.png (box + confidence + footer).

Flow: Image → YOLO → crops + JSON → ticket/evidence page (can extend to PDF if needed).



Image: illegal-parking-101-_png.rf.7f8fc09a720db55d002e189e495c9c6a.jpg | Detection: illegal parking | Conf:

Limitations & Next Steps

Limitations

Single class; no per-violation labels, so we can't identify specific infractions.

Centered framing bias; wide-FOV or complex street scenes may need additional scene-level negatives/augs.

Metrics may benefit from dataset format (car-centered, consistent framing); real-world deployment needs more diverse negatives and condition-specific testing.

Next steps

Acquire/label violation-type data (hydrant, crosswalk, signage) to move beyond binary.

Add scene-level negatives and condition-specific augmentations (night, rain).

Evaluate with alternative backbones (yolov8s) if latency permits; tune conf/NMS for deployment.

Integrate a PDF evidence generator for a polished workflow.

Self-Critique

Data is the main bottleneck: no violation-type labels; scope narrowed to binary.

Framing bias: car-centered negatives improved precision but may hurt generalization; need wider-scene negatives.

High scores partly reflect consistent framing and single class; real-world tests should include more varied backgrounds and lighting.

Model choice is adequate; gains now depend more on data quality than epochs or minor tweaks.